It's Not How Many, It's Where: Optimizing Routing Selection for Filtered Vector Search

Sanjay Mishra
Independent Researcher
Raleigh, NC, USA
sanmish4@icloud.com

ABSTRACT

A filtered vector query asks for the items most similar to a query, but only among those that satisfy a condition, such as "the ten papers closest to this one, published in physics". A database can answer it in three ways: filter first and then compare vectors, search a similarity graph and then filter, or filter during the graph search. The fastest choice varies sharply from query to query. Today's optimizers decide mainly from *how many* rows match the filter. We show that what matters is *where* the matching rows are. If they lie near the query, a graph search finds them quickly; if they lie far away, the search wastes time and filtering first is faster. For example, two real arXiv queries whose filters each match about 6% of the table need opposite plans: the plan that is best for one is 48× slower than necessary for the other.

We present *HyperVector-Optimizer* (HVO). HVO estimates how far a search must travel to find enough matches, using a small summary of the data that never touches the index, and then picks the plan that is safest given its uncertainty. HVO runs both in a standalone benchmark and inside PostgreSQL with `pgvector`. On one million arXiv papers, it cuts mean latency by 43% (95% CI 34–51%) compared with the best tuned rule based on filter size, and reduces tail latency up to fourfold. On Amazon products and synthetic data, the gain in mean latency is not statistically clear. In PostgreSQL, `pgvector` with default settings reaches a mean recall of only 0.540 on arXiv (it returns about half of the correct results); HVO returns more correct results than forcing `pgvector`'s iterative scan on every query, and is faster. We release the code and both real benchmarks.

PVLDB Reference Format:
Sanjay Mishra. It's Not How Many, It's Where: Optimizing Routing Selection for Filtered Vector Search. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/sanmish4ds/HyperVectorOptimizer.

1 INTRODUCTION

Search applications increasingly ask a database for items that are similar to something, but only among items that meet some conditions. A chatbot may look for documents similar to a question, but only from a certain team; a shop may look for products similar to one a customer viewed, but only in stock and under a price limit [1, 22, 29, 32]. Each item is stored as an *embedding*—a list of numbers that captures its meaning—next to ordinary columns such as category or year. A typical query asks for the ten papers most similar to a given one, but only among published physics papers:

```
SELECT id FROM papers
WHERE published AND archive IN ('physics', ...)
ORDER BY embedding <#> q LIMIT 10;
```

In general, such a query combines range conditions, `IN` lists and conditions on a joined table with an ordering by similarity to the query vector $\mathbf{q}$, and asks for the top k results.

Many systems support such queries. Some are ordinary databases that store embeddings next to regular columns, such as PostgreSQL with `pgvector` [13] or DuckDB with `vss` [6]. Others are dedicated vector databases, such as Milvus [27]. In every system, there are three basic ways to answer the query:

- *Filter first* (Π_{pre}). Find every row that passes the filter, compare each one with the query, and keep the closest k. This always gives the exact answer, but it may compare tens of thousands of rows to return just ten.
- *Search first* (Π_{post}). Use a similarity index (HNSW [18]) to visit rows in order of similarity, starting with the closest, and check each against the filter until k have passed. This is how `pgvector`'s iterative scan and VBase [32] work. It is very fast when matching rows are close to the query, but slow when they are far away, and it can miss some answers.
- *Filter while searching* (Π_{fused}). Walk the index as before, but step only onto rows that pass the filter [22, 27]. This is often fast, but it can miss answers when few matching rows lie near the query.

The best choice can differ greatly from one query to the next. Figure 1 shows two real queries from our arXiv benchmark. Query A looks for the ten papers most similar to a computer-science paper, *"Deep Active Learning for Dialogue Generation"*, but only among physics papers. Query B looks for the ten papers most similar to a physics paper, *"SIS Epidemic Spreading with Correlated Heterogeneous Infection Rates"*, but only among published physics papers. Both filters keep about 6% of the one million papers (5.9% and 6.6%). Database optimizers choose a plan mainly from this kind of number: the share of rows that pass the filter. A rule based only on this share must treat the two queries the same way. The best such rule we could tune filters both first. That is the right choice for Query A (4.03 ms), but 48 times too slow for Query B, which searching first answers in just 0.11 ms. No rule based only on *how many* rows match can get both queries right (Section 3.3).

What separates the two queries is not how many papers match, but *where* they are. A paper on dialogue generation sits among

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

computer-science papers, far from physics papers. A paper on epidemics, filed under physics, sits among physics papers. A search that starts at the query and moves outward is cheap when matching papers are nearby and expensive when they are far away. For Query B, the search passes only 34 papers before it has found ten matches; for Query A it must pass 13,490. We call the share of rows that pass the filter the *global selectivity*, and k divided by the number of rows the search must pass the *local selectivity*. Both queries have a global selectivity of about 6%, but the local selectivity is 29.4% for Query B and only 0.0741% for Query A.

Recent work recognizes that global selectivity is not enough. Exqutor estimates how many results a vector search will find by probing the index or by sampling [14], and benchmark studies show that performance swings widely when filters and similarity are related [1, 12, 25]. Three questions remain open: how to define the local selectivity that decides the plan, how to estimate it quickly without touching the index, and how to choose a plan safely, inside a real database, when the estimate is wrong.

This paper presents *HyperVector-Optimizer* (HVO), which chooses each query's plan from its local selectivity. HVO estimates local selectivity with a small summary of the data, the *Polytope-Ball Sketch* (HPBS), without touching the index. It predicts each plan's running time from measurements taken on the actual machine, and it picks the plan that stays fast even if its estimate is somewhat wrong. For Query A, HVO estimates 0.0578% (true: 0.0741%) and filters first; for Query B it estimates 12.4% (true: 29.4%) and searches first. Both are the fastest plans that return the correct answer. We also prove that HVO never sends a query whose best plan is filtering first to a slower plan, as long as its uncertainty range covers the true value, and that any rule based only on global selectivity can become arbitrarily slow as the table grows.

We test HVO on two real datasets, one million arXiv papers and one million Amazon products, and on two synthetic ones. To know the best plan for certain, we run every plan for every test query. Figure 1 is not a rare case: on the arXiv queries, HVO and the best global rule choose different plans for 15% of queries, and HVO's choice is faster in 89% of those. Overall, HVO cuts average query time on arXiv by 43% (95% confidence interval 34–51%) and cuts the time of the slowest queries by up to four times. On Amazon it speeds up the slowest queries, but its gain in average time is not statistically clear; on synthetic data the averages tie. Inside PostgreSQL, `pgvector` with default settings and Exqutor reach a mean recall of only about 0.540 on these queries, returning about half of the correct results. HVO returns more correct results than forcing `pgvector`'s iterative scan on every query, and is faster. HVO does not help when only one result is requested ($k = 1$), because planning then takes longer than the query itself.

Our contributions are:

- **Local selectivity.** A definition of local selectivity based on the k-th match, and a sketch that estimates it without touching the index (Sections 3 and 4.2).
- **Robust plan choice.** A way to choose plans safely using measured costs, with proven guarantees (Sections 4.4–4.6; proofs in the extended version [21]).
- **PostgreSQL integration.** HVO as a planner extension for PostgreSQL and `pgvector` (Section 4.7).
- **Evaluation and benchmarks.** An evaluation in which every plan runs for every query, and two real-data benchmarks released with exact answers [20] (Section 5).

2 BACKGROUND AND RELATED WORK

Databases with vector search. Several engines combine vector search with ordinary queries. AnalyticDB-V [29] chooses among several hybrid plans with a cost model. VBase [32] keeps pulling results from an HNSW index until enough of them pass the filter. `pgvector` [13] adds an iterative index scan to PostgreSQL that works the same way, and DuckDB's `vss` [6] adds a vector index to DuckDB. Milvus [27] turns the filter into a bitmap of matching rows that the vector search consults. *Limitation:* these systems choose a plan with fixed rules or with costs based on how many rows match the filter, so they cannot tell a filter whose matches surround the query from one whose matches lie far away.

Indexes built for filters. A second line of work builds the filter into the index itself. Filtered-DiskANN [8] and NHQ [28] add labels to the graph during construction; HQANN [30] mixes attribute and vector distances into one score; SeRF [33] and its dynamic successor [23] build special graphs for range filters on one attribute; and ACORN [22] adds extra links to HNSW so that the graph stays connected when only matching rows may be visited. Surveys and benchmarks [1, 12, 17, 25] find that no single index is best across all filters. *Limitation:* each index commits to one way of filtering. None decides, query by query, whether another available plan would be cheaper, so the optimizer still has to choose.

Estimating result sizes. To choose plans, databases estimate how many rows a condition will match. Classic estimators use histograms and wavelets [9, 19], compact sketches such as HyperLogLog and k-minimum values [3, 7], or learned models [10, 15, 31]. For similarity queries, estimators use locality-sensitive hashing [4, 26] or reference objects [16]. Exqutor [14] is closest to our work: it gives a database optimizer estimates of how many results a vector search returns, by probing the index when one exists and by sampling otherwise. *Limitation:* classic estimators assume that the filter has nothing to do with the vectors; similarity estimators count matches within a fixed distance rather than the distance to the k-th match, which is what decides the cost of a graph search; and probing the index costs time during planning and cannot see matches that lie beyond the probed neighbors.

Summary. HVO differs from the closest work in three ways: it estimates how many matches lie near the query from a summary, without touching the index; it estimates exactly the distance to the k-th match, which decides graph-search cost; and it combines the estimate with measured plan costs, proven guarantees and a PostgreSQL integration. We compare against an Exqutor-style index probe and a sampling estimator in our benchmark, and against Exqutor itself, built from its released code, inside PostgreSQL.

3 FILTERED VECTOR SEARCH, BY EXAMPLE

This section explains the problem with the two queries of Figure 1 before we formalize it. We first fix the setting (Section 3.1), then follow the two queries through the three plans (Section 3.2), and finally show why the share of matching rows cannot predict the best plan (Section 3.3).

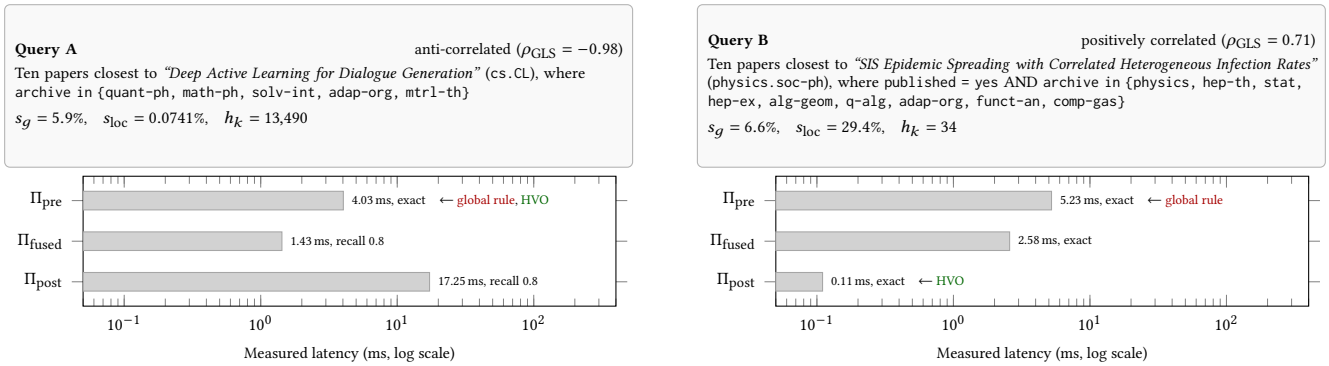


Figure 1: A real example from the ARXIV-384 benchmark ($k = 10$). The two filters match about the same share of the table, so a router that sees only global selectivity sends both queries to the same plan (here filter first, because both are below its tuned threshold of 7.9%). Query B's matches surround the query paper, and searching first answers it 48× faster; Query A's matches lie far away, and searching first must pass 13,490 papers. HVO estimates the local selectivity and routes both queries to their fastest plan that returns the correct answer. "recall": share of the true top 10 returned by the graph plans. The pair was selected from the test split by a fixed rule to illustrate the failure mode (Section 3); Section 5 reports how often it occurs.

3.1 Setting and Assumptions

A table R holds N rows. Each row has a vector $\mathbf{v}_i$ of length one in $\mathbb{R}^d$ and ordinary attributes $\mathbf{a}_i$, and it may link to a small second table through a foreign key. A query $(\mathbf{q}, k, \mathcal{P})$ asks for the k rows most similar to $\mathbf{q}$ (largest inner product) among those that satisfy the filter $\mathcal{P}$. The filter is a combination (AND) of range conditions, `IN` lists and conditions on the linked table. In ARXIV-384, R holds one million papers with 384-dimensional embeddings; the attributes include the paper's archive, whether it is published, its year and its number of authors, and the foreign key points to its primary category.

We assume that every plan keeps working until it has k answers (or runs out of budget), so how long a plan takes does not depend on what the optimizer believed; the optimizer only decides *which* plan runs. This lets us measure the best plan of every query: we run all three plans and take the fastest one that returns at least 9 of the true top 10 (Recall@$k \geq 0.9$). We call this the *oracle*. The optimizer itself must choose before running anything, from estimates computed in a small fraction of the query time. Plans run on one thread. Everything the optimizer learns—cost constants, the calibrator and the uncertainty range—is fitted on training queries and tested on different queries.

3.2 Three Plans, Two Queries

Table 1 follows Query A and Query B through the three plans. Section 4.3 describes the plans in full; here we only need what each one does.

Filter first checks the filter on all one million rows, which takes a few milliseconds for any query, and then compares the query with every matching vector: 58,544 for Query A and 66,356 for Query B. Its cost depends only on how many rows match, and its answer is always exact. It takes 4.03 and 5.23 ms.

Search first starts at the query's position in the HNSW graph and visits rows roughly from most to least similar, checking each against the filter, until k have passed. For Query B the tenth match is the 34-th most similar paper, so the search stops almost immediately (0.11 ms). For Query A it is the 13,490-th: the search must pass more than ten thousand computer-science papers before it reaches enough physics papers, and it takes 17.25 ms. Because it walks a graph instead of checking every row, it can also miss correct answers (Recall@10 0.8).

Table 1: The running example in numbers ($k = 10$). Latency in ms (Recall@10 in parentheses when below 1). "Global": the plan chosen by the best tuned global-selectivity router; "HVO": the plan HVO chooses; "Oracle": the fastest plan with Recall@10 ≥ 0.9.

	Query A (anti)	Query B (positive)
Global selectivity s_g	5.9%	6.6%
h_k (tuples before 10th match)	13,490	34
Local selectivity k/h_k	0.0741%	29.4%
Π_{pre} (pre-filter)	4.03	5.23
Π_{fused} (ACORN-1)	1.43 (0.8)	2.58
Π_{post} (post-filter)	17.25 (0.8)	0.11
Oracle / Global / HVO	Π_{pre} / Π_{pre} / Π_{pre}	Π_{post} / Π_{pre} / Π_{post}

Filter while searching (ACORN-1 [22]) checks the filter first, like filter first, but then walks the graph stepping only onto matching rows. It is fast on both queries (1.43 and 2.58 ms), but on Query A it returns only 0.8 of the true top 10.

So the fastest plan that returns a correct answer is filter first for Query A and search first for Query B. Choosing wrongly costs 4× on Query A and 48× on Query B.

3.3 Understanding Why the Share of Matches Is Not Enough

The example suggests that the right plan depends on how far the search must travel before it meets k matches. We now make this precise, and then show that the share of matching rows does not carry this information.

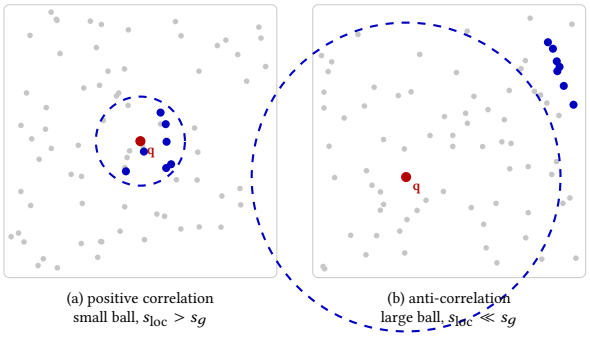


Figure 2: Schematic of local selectivity (illustration, not data). Gray: tuples that fail the filter; blue: matches; red: the query. The dashed ball is the smallest ball around q that contains k matches, and h_k is the number of tuples inside it. Both panels have the same global selectivity. A post-filter must pass every tuple in the ball, so its cost follows $h_k = k/s_{\text{loc}}$, not s_g.

How far the search must go. Imagine listing all rows from most to least similar to the query. Search first walks down this list, so its work is set by how far down the k-th matching row appears. We call this position h_k and define local selectivity from it:

Definition 3.1 (k-th-match local selectivity). Order R by distance to $\mathbf{q}$. Let $h_k(\mathbf{q}, \mathcal{P})$ be the number of rows whose distance to $\mathbf{q}$ is at most that of the k-th closest row that satisfies $\mathcal{P}$. The local selectivity is $s_{\text{loc}}(\mathbf{q}, \mathcal{P}) = k/h_k(\mathbf{q}, \mathcal{P})$.

In words, s_{loc} is the share of matching rows inside the smallest ball around $\mathbf{q}$ that contains k matches. If the filter had nothing to do with the vectors, matches would be spread evenly, so $h_k \approx k/s_g$ and $s_{\text{loc}} \approx s_g$, where s_g is the global selectivity (the share of all rows that match). An earlier version of this work used the share of matches inside a ball of fixed size; that predicted search cost poorly when matches were far from the query, because the search must then go far beyond the fixed ball. Definition 3.1 measures the distance the search actually travels. To describe how strongly a filter and the query's neighborhood are related, we use the global-local selectivity coefficient [1]

$$\rho_{\text{GLS}} = \tanh\left(\tfrac{1}{2} \ln \frac{s_{\text{loc}}(1 - s_g)}{s_g(1 - s_{\text{loc}})}\right) \in (-1, 1). \tag{1}$$

It is positive when matches crowd around the query and negative when they lie far away. We call a query *anti-correlated* if $\rho_{\text{GLS}} < -0.3$, *positively correlated* if $\rho_{\text{GLS}} > 0.3$, and *independent* otherwise.

Figure 2 illustrates the definition. When matches lie near the query (left), the smallest ball holding k of them is small and $s_{\text{loc}} > s_g$. When they lie far away (right), the ball must grow until it reaches them, and $s_{\text{loc}} \ll s_g$. In the example, the two queries have almost the same global selectivity (about 6%), but their local selectivities differ by a factor of four hundred.

It is not just the example. Among the 76 ARXIV-384 test queries whose global selectivity is within 15% of the example's, local selectivity ranges from 0.031% to 56%, and the fastest correct plan is filter first for 18% of them, search first for 72% and filter while searching for 9%. Any rule that picks a plan from the global selectivity alone must treat all of them the same way.

Where each plan wins. Writing the plan costs of Section 4.4 as functions of s_g and s_{loc} shows that filtering first is cheapest only below a boundary $s_g^*(s_{\text{loc}})$. Because checking the filter column by column already costs time proportional to the table size, a graph plan wins for every s_g once s_{loc} exceeds roughly 1–2%, so the boundary is nearly flat and the global selectivity says little about the right plan. The extended version [21] states and proves this as a theorem and plots the boundary.

4 HYPERVECTOR-OPTIMIZER

We now describe how HVO makes the decision of Figure 1 automatically, in a small fraction of the query's running time.

4.1 Overview

Before a query runs, HVO answers one question: *how far must the search go to find k matches?* If the answer is "not far", searching first is fast; if it is "very far", filtering first is usually better. HVO answers the question in four steps, shown in Figure 3.

Step 1: Read the filter. HVO turns the query's filter into a simple form that it can check quickly. For Query B, "published physics papers" becomes `published = yes` and `archive` is one of nine values.

Step 2: Estimate how far the search must go. HVO keeps a small summary of the data, the *Polytope-Ball Sketch* (HPBS, Section 4.2). The summary splits the table into about a thousand groups of similar rows and remembers, for each group, how spread out its vectors are and how many of its rows have each attribute value. From it, HVO estimates the local selectivity without touching the index or the table. For Query B it estimates 12.4% (true value 29.4%); for Query A, 0.0578% (true value 0.0741%).

Step 3: Predict each plan's time. HVO predicts how long each plan would take (Section 4.4). The predictions come from timing the plans on the actual machine, not from counting operations. For Query B, it predicts about 0.16 ms for searching first and 4.40 ms for filtering first.

Step 4: Choose a plan that is safe even if the estimate is wrong. An estimate can be off by a factor of two or three, so HVO considers a range of possible values instead of a single number, and picks the plan whose *worst case* within that range is fastest (Section 4.5). For Query B, even in the worst case searching first is predicted to take 0.63 ms, much less than filtering first (4.43 ms), so HVO searches first. For Query A, searching first could take up to 58.55 ms in the worst case, while filtering first takes about 3.54 ms whatever the truth, so HVO filters first.

The chosen plan then runs. Planning reads only the summary, never the index or the data. The summary, the time predictions and the size of the safety range are prepared in advance from training queries, and the same planning code runs in our benchmark and inside PostgreSQL (Section 4.7).

4.2 Estimating How Far the Search Must Go

HPBS answers one question for each query: *how many rows must the search look at before it finds k that pass the filter?* If we list all rows from most to least similar to the query, the answer is how

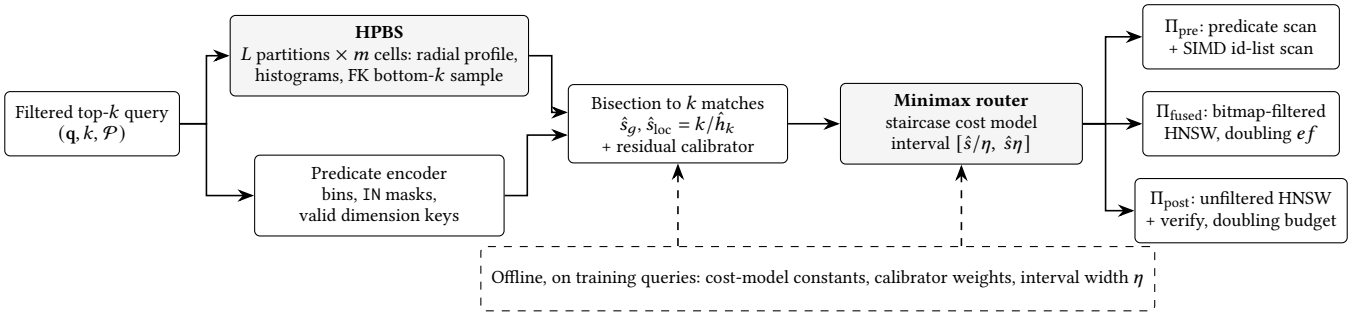


Figure 3: Architecture of HVO. Planning reads only the sketch, never the vector index. The router evaluates a calibrated cost model at both ends of the estimator's uncertainty interval and runs the plan with the smallest worst-case cost.

far down the list the k-th matching row appears: h_k, with local selectivity k/h_k (Definition 3.1). For Query B, $h_k = 34$; for Query A, $h_k = 13{,}490$. HPBS must answer quickly, without running the search and without touching the index or the table, so it works from a small summary of the data prepared in advance.

The idea. HPBS keeps a coarse map of the data. It splits all vectors into $m = 1024$ groups (*cells*) of similar items with k-means clustering, and remembers three things about every cell b:

- *Where the cell is and how spread out it is:* its center $\mathbf{c}_b$, its size N_b, and 16 quantiles $\rho_{b,1}, \ldots, \rho_{b,16}$ of its members' squared distances to the center.
- *What its members look like:* for each attribute, the share of members with each value or in each range (for example, 40% physics, 15% from 2010–2015).
- *Which linked rows its members point to:* up to $k_{\text{kmv}} = 64$ foreign keys, chosen by hashing, with the number of members that carry each one ($\mathcal{K}_b$).

Estimating for a query. HPBS asks every cell two questions. First, *how many of your members lie within distance r of the query?* With $\delta_b = \|\mathbf{q} - \mathbf{c}_b\|^2$, and because in high dimensions a member's offset from its center is nearly at a right angle to $\mathbf{q} - \mathbf{c}_b$ (so $\|\mathbf{q} - \mathbf{x}\|^2 \approx \delta_b + \|\mathbf{x} - \mathbf{c}_b\|^2$), the answer is

$$\hat{n}_b(r) = \frac{N_b}{16}\left|\{j : \delta_b + \rho_{b,j} \le r\}\right|. \tag{2}$$

Second, *what share of them pass the filter?* HPBS reads each condition's share $\hat{\sigma}_b^{(j)}$ from the stored statistics. For a condition on the linked table, it evaluates the condition on that small table to get the set $\mathcal{V}$ of valid keys, and estimates the share of the cell's rows that point to a valid key as

$$\hat{\sigma}_b^{\text{join}} = \frac{\sum_{h \in \mathcal{K}_b} f_b(h)\, \mathbb{I}[h \in \mathcal{V}]}{\sum_{h \in \mathcal{K}_b} f_b(h)}, \tag{3}$$

where $f_b(h)$ is the number of the cell's rows with key h. Assuming the conditions are independent *within* a cell, the cell's matching share is $\hat{\sigma}_b = \prod_j \hat{\sigma}_b^{(j)} \cdot \hat{\sigma}_b^{\text{join}}$. Multiplying the two answers and adding over all cells gives the expected number of matches within distance r, $\hat{M}(r) = \sum_b \hat{n}_b(r)\hat{\sigma}_b$. HPBS finds the distance $\hat{r}$ at which $\hat{M}(\hat{r}) = k$ by repeatedly halving the search range (26 steps), and returns

$$\hat{s}_{\text{loc}} = \min\Big(1,\ \frac{k}{\sum_b \hat{n}_b(\hat{r})}\Big), \qquad \hat{s}_g = \frac{1}{N}\sum_b N_b \hat{\sigma}_b. \tag{4}$$

Table 2: Algorithm 1: HPBS query-time estimation (one partition; the sketch returns the median over L).

Input: $\mathbf{q}$, k, predicate $\mathcal{P}$, partition $\{\mathbf{c}_b, N_b, \rho_{b,\cdot}, \text{hist}_b, \mathcal{K}_b\}_{b=1}^{m}$.
1: Encode $\mathcal{P}$: boundary bins and offsets of each range, bitmask of each IN list, valid key set $\mathcal{V}$ of the dimension predicate.
2: $\boldsymbol{\delta} \leftarrow \|\mathbf{c}\|^2 - 2\,\mathbf{C}\mathbf{q} + 1$ ▷ one sgemv
3: **for** each cell b: $\hat{\sigma}_b \leftarrow \prod_j \hat{\sigma}_b^{(j)} \cdot \hat{\sigma}_b^{\text{join}}$ (Eq. (3)); $\hat{s}_g$ += $N_b\hat{\sigma}_b/N$.
4: $C \leftarrow \{b : \hat{\sigma}_b > 0\}$; $lo \leftarrow 0$; $hi \leftarrow \max_{b \in C} \delta_b + \rho_{b,16}$.
5: **repeat** 26 times: $r \leftarrow (lo + hi)/2$; **if** $\sum_{b \in C} \hat{n}_b(r)\hat{\sigma}_b < k$ **then** $lo \leftarrow r$ **else** $hi \leftarrow r$; $C \leftarrow \{b \in C : \delta_b \le hi\}$.
6: **return** $\hat{s}_g, \hat{s}_{\text{loc}} \leftarrow \min(1, k/\sum_b \hat{n}_b(hi))$.

Algorithm 1 lists the steps.

Why this captures correlation. HPBS assumes independence only *inside* a cell. Across cells it keeps the real picture: physics papers are common in some cells and absent from others. This is exactly the information that is lost when an optimizer counts matches over the whole table.

Combining and correcting. HPBS builds $L = 4$ such maps with different random starts and returns the median of their four estimates, which smooths out unlucky groupings. A small neural network with 497 parameters, trained only on the training queries, then corrects the remaining bias: $\hat{s}_{\text{loc}} \leftarrow \text{clip}(\hat{s}_{\text{loc}}^{\text{raw}} e^{f_\Theta(\mathbf{z})}, k/N, 1)$, where $\mathbf{z}$ holds twelve features of the raw estimate and the query [21]. It mainly helps when a query lies very close to a cell's center, or when the independence assumption inside a cell is poor. The correction takes about 4.0 μs and mostly removes large errors: on ARXIV-384 the 99th-percentile Q-error falls from 9.95 to 6.17.

Example. For Query A, the filter selects physics papers, but the cells around this computer-science paper contain almost none, so the estimated distance must grow until it reaches cells where physics papers are common. HPBS estimates a local selectivity of 0.0578%, close to the true 0.0741%. For comparison, looking at the index's 256 nearest neighbors finds *no* physics paper at all, so that method can only report "less than 0.195%" (Section 5.2). For Query B, the cells near the query contain many published physics papers, and HPBS estimates 12.4% (true value 29.4%). Both estimates are within a factor of 2.5 of the truth, which is enough for HVO to pick the right plan.

Where the errors come from. Let $n_b(r)$ and $M_b(r)$ be the true numbers of rows and of matches of cell b within squared distance r, $\sigma_b(r) = M_b(r)/n_b(r)$, and $h(j)$ the number of rows within the

distance of the j-th closest match, so $h_k = h(k)$. Proposition 4.1 splits the estimation error into two parts: errors in *how many* rows of each cell lie within a given distance (α), and errors in *what share* of them pass the filter (β).

Proposition 4.1 (Deterministic error decomposition). *Suppose that for every r and b the geometric and cell estimates satisfy*

$$\hat{n}_b(r) \in \big[n_b(r)/\alpha,\ \alpha n_b(r)\big], \qquad \hat{\sigma}_b \in \big[\sigma_b(r)/\beta,\ \beta\sigma_b(r)\big].$$

Then

$$\text{Q-error}(\hat{s}_{loc}, s_{loc}) \le \alpha \cdot \psi(\alpha\beta), \quad \psi(t) = \max\Big(\frac{h(\lceil tk\rceil)}{h(k)},\ \frac{h(k)}{h(\lfloor k/t\rfloor)}\Big).$$

If matches are locally uniform around $\mathbf{q}$ *($h(j) \propto j$), then $\psi(t) \approx t$ and the Q-error is at most $\alpha^2\beta$.*

Smaller cells reduce α, and finer statistics reduce part of β. The rest of β comes from relationships *inside* a cell, which a summary of this kind cannot see. That is why HPBS is less accurate on AMAZON-384, where each filter is built from the query product itself and matches cluster tightly around the query (Section 5.2). The proposition explains where errors come from; it does not promise an error bound for every dataset, and we prove nothing about the calibrator. The proof is in the extended version [21].

Size and cost. For one million vectors, HPBS takes 5.1, 9.1 and 15.1 MiB for 128-, 384- and 768-dimensional vectors (0.52–1.05% of the size of the vectors themselves). It is built in 10–24 s, against 93–214 s for the HNSW index on 10 threads. Adding a row updates one cell in each map; we have not evaluated workloads with frequent updates.

4.3 The Three Plans

All plans run on one thread. *Filter first* (Π_{pre}) checks the filter column by column and compares the query with each matching row in a C loop (NEON SIMD). *Search first* (Π_{post}) is a *resumable* scan over a faiss [5] HNSW graph ($M = 32$, $ef_{\text{construction}} = 200$) in the spirit of `pgvector`'s iterative scan: it returns rows in batches of $k, k, 2k, 4k, \ldots$ in roughly decreasing similarity, checks each batch against the filter, and continues from where it stopped until k rows have passed (at most 2^{17}). *Filter while searching* (Π_{fused}) is ACORN-1 [22] ($M = 32$, $M_\beta = 64$), which doubles its search budget from 64 until it has k results and a doubled budget returns the same set. Running ACORN at one million vectors required fixing two memory bugs in its reference code; the fix has been submitted upstream. Because each plan keeps going until it has its answer, its running time does not depend on the optimizer's estimate.

4.4 Predicting Plan Costs from Measurements

A cost model answers the question: *if the local selectivity were s, how long would each plan take?* A common shortcut is to count operations, such as distance computations. We instead fit each plan's cost to running times measured on the target machine, because the same number of distance computations costs very different amounts in a sequential scan and in a random walk over a graph (Section 5.6 shows that counting makes the routing 1.6–6.3× slower).

Graph plans grow in rounds. A graph plan doubles its budget in each round ($e_i = e_0 2^i$) and stops once the budget reaches what the query needs, v. It therefore runs $j(v) = \lceil \log_2(v/e_0)\rceil^+$ rounds, capped at the budget limit. If a round with budget e costs $b_0 + b_1 e^\gamma$, the plan's cost is a "staircase" in v:

$$\mathrm{St}(v; b_0, b_1, \gamma) = \sum_{i=0}^{j(v)} \big(b_0 + b_1 (e_0 2^i)^\gamma\big). \tag{5}$$

The three costs. Let $C_{\text{rel}} = c_0 + c_1 N|\mathcal{P}| + c_2 N\,\mathbb{I}[\text{join}]$ be the cost of checking the filter over all rows. Then

$$C(\Pi_{\text{pre}}) = C_{\text{rel}} + a_0 + c_{\text{scan}}\, s_g N, \tag{6}$$

$$C(\Pi_{\text{post}}) = \mathrm{St}\big(k/s_{\text{loc}};\ \theta_{\text{post}}\big), \tag{7}$$

$$C(\Pi_{\text{fused}}) = C_{\text{rel}} + \mathrm{St}^{+1}\big(c_f (k/s_{\text{loc}})^{\beta_f};\ \theta_{\text{fused}}\big), \tag{8}$$

where St^{+1} adds ACORN's final confirmation round. In words: filtering first pays for checking the filter plus one comparison per matching row; searching first needs exactly $h_k = k/s_{\text{loc}}$ rows (Definition 3.1); and filtering while searching pays for checking the filter plus a graph walk whose budget grows faster than h_k, which the exponent β_f captures.

Fitting the constants. All constants are fitted on the training queries from their *true* selectivities and measured times, the linear ones by non-negative least squares and the staircase ones by non-linear least squares in log space ($\gamma \in [0.3, 2.5]$). Exponents above one are common, because a graph search's priority queues make large budgets disproportionately expensive. ACORN's cost could not be fitted by a smooth staircase, so we model it with a monotone (isotonic) curve in h_k.

Example. At the true selectivities, the model predicts 3.54 ms (filter first), 8.67 ms (filter while searching) and 16.55 ms (search first) for Query A, against measured times of 4.03, 1.43 and 17.25 ms; and 4.40, 4.05 and 0.16 ms for Query B, against 5.23, 2.58 and 0.11 ms. The model picks the oracle's plan for both queries. It overestimates ACORN on Query A, where ACORN is in fact fast but returns only 0.8 of the correct answers; ACORN's cost is the hardest to model (Section 5.3).

4.5 Choosing a Plan Robustly

An estimate can be off by a factor of two or three, and the damage of a wrong estimate is lopsided. Filtering first costs the same whatever the local selectivity, while a graph plan's cost explodes when the local selectivity is lower than believed. HVO therefore asks, for each plan, *"how slow could this plan be if the truth were anywhere in my uncertainty range?"*, and picks the plan whose worst case is fastest.

Point and minimax routing. Point routing trusts the estimate and picks $\arg\min_\Pi C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}})$. *Minimax routing* considers the range $I = [\hat{s}_{\text{loc}}/\eta,\ \hat{s}_{\text{loc}}\eta]$ and picks

$$\begin{aligned}\hat{\Pi} &= \arg\min_{\Pi}\ \max_{s\in I} C(\Pi; \hat{s}_g, s) \\ &= \arg\min_{\Pi}\ \max\big(C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}/\eta),\ C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}\eta)\big).\end{aligned} \tag{9}$$

The second line holds because every plan's cost only goes up as s_{loc} goes down, so the worst case is at one end of the range. We set η to the value that covers 90% of the estimator's errors on the training queries, fixed before looking at test data ($\eta = 1.87$ and 2.10 on the synthetic datasets). Because the cost of filtering first does not depend on s_{loc}, minimax routing prefers it whenever the estimate is uncertain.

Example. On ARXIV-384, $\eta = 2.52$, so Query B's range is [4.93%, 31.2%]. Even at the bad end of that range, searching first is predicted to take 0.63 ms, against 4.43 ms for filtering first, so HVO searches first. Query A's range is [0.023%, 0.145%]. At its bad end, searching first could take 58.55 ms and filtering while searching 8.67 ms, while filtering first takes 3.54 ms whatever the truth, so HVO filters first. Both are the oracle's choices.

Recall-aware routing. Graph plans can miss correct answers. For each graph plan we fit a small logistic model, on the training queries, that predicts the chance of getting at least 9 of the top 10 from values known at planning time (the two estimates, the number of filter conditions and whether there is a join). The recall-aware router allows a graph plan only if this chance is at least 0.9, and otherwise filters first. This trades speed for completeness, and we report both variants.

Native planner. The whole decision—sketch estimate, calibrator, recall check and minimax costing—runs in one C function, `hvo_plan`, shared by the benchmark and the PostgreSQL extension. It takes the encoded filter and the query vector and returns the plan.

4.6 Guarantees

The example shows minimax routing making the right choice on two queries. The results below say when it is guaranteed to, and how much an estimation error of a given size can cost. Three facts drive them. A graph plan doubles its budget in rounds, so a small estimation error changes its cost by at most a few rounds (Lemma 4.2). Filtering first costs the same whatever the local selectivity, so a minimax router never gives up a query whose best plan is filtering first (Theorem 4.4). And a rule that sees only the share of matching rows cannot tell the two panels of Figure 2 apart, so it can be made arbitrarily slow (Proposition 4.5).

We measure the cost of a wrong choice by the *regret* $\mathcal{R} = C(\hat{\Pi}; s)/C(\Pi^*; s)$: the cost of the chosen plan divided by the cost of the best plan Π^* at the true selectivities $s = (s_g, s_{\text{loc}})$. A regret of 1 means the best plan was chosen. The results concern the cost model of Section 4.4; measured regret also includes the model's own error, which we report in Section 5. All statements hold query by query; proofs are in the extended version [21].

Lemma 4.2 (Sensitivity of staircase plans). *Let a plan's cost be $A + \mathrm{St}(c\,(k/s)^{\beta}; b_0, b_1, \gamma)$ with $A, b_0, b_1 \geq 0$ and $\gamma > 0$. If $\hat{s}/s \in [1/\eta, \eta]$, then its cost at $\hat{s}$ is within a factor $\Lambda_{\beta,\gamma}(\eta)$ of its cost at s, where*

$$\Lambda_{\beta,\gamma}(\eta) = 1 + \delta\, 2^{\gamma\delta}, \qquad \delta = \lceil \beta \log_2 \eta \rceil.$$

Let $\Lambda = \max(\eta_g, \Lambda_{1,\gamma_{\text{post}}}(\eta), \Lambda_{\beta_f,\gamma_{\text{fused}}}(\eta))$, where η_g bounds the multiplicative error of $\hat{s}_g$, which enters $C(\Pi_{\text{pre}})$ affinely.

Theorem 4.3 (Point routing). *If $\hat{s}_g/s_g \in [1/\eta_g, \eta_g]$ and also $\hat{s}_{loc}/s_{loc} \in [1/\eta, \eta]$, then point routing has $\mathcal{R} \leq \Lambda^2$. If, in addition, $\hat{s}_g = s_g$ and either $\hat{\Pi}$ or Π^* is Π_{pre}, then $\mathcal{R} \leq \Lambda$.*

Theorem 4.4 (Minimax routing). *Suppose $\hat{s}_g = s_g$ and $s_{loc} \in I = [\hat{s}_{loc}/\eta, \hat{s}_{loc}\eta]$. Then minimax routing (Eq. (9)) satisfies: (i) if $\Pi^* = \Pi_{pre}$, then $\mathcal{R} = 1$; (ii) in all cases $\mathcal{R} \leq \Lambda(\eta^2)$.*

Proposition 4.5 (Global-selectivity routing). *Let a router choose its plan as any function of s_g alone. Let graph-plan costs satisfy $C_G(1) \leq C_1$ and $C_G(k/N) \geq bN^{\gamma}$ for constants $C_1, b, \gamma > 0$, and let $C(\Pi_{pre}) = a + c' s_g N$, which holds when predicates are evaluated through an index at a cost proportional to their output. Then for N large enough that $k \leq \sqrt{bN^{\gamma}C_1/c'}$, its worst-case regret over instances with that N is at least $\sqrt{bN^{\gamma}/(c'C_1)}\,/\,(1 + o(1))$, that is, $\Omega(N^{\gamma/2})$.*

4.7 Integration into PostgreSQL

We implemented HVO as a PostgreSQL 16 extension (`hvo_pg`) for `pgvector` 0.8.7, using PostgreSQL's `planner_hook`, which lets an extension see every query before it is planned. For a query that orders by `embedding <#> q` with a `LIMIT`, the hook does three things:

- It reads the query vector, k, and the filter: ranges, equalities and `IN` lists on the main table, and conditions on the linked table written as `fk IN (SELECT id FROM dims WHERE ...)`.
- It encodes the filter for the summary, evaluating conditions on the linked table against an in-memory copy of that small table.
- It calls `hvo_plan` and steers PostgreSQL's own planner toward the chosen plan with standard settings, as the `pg_hint_plan` extension does: it turns index scans off to filter first, and turns sequential and bitmap scans off to use `pgvector`'s iterative HNSW scan.

`pgvector` cannot filter while searching, so in PostgreSQL the choice is between two plans. The model file holds the summary, the calibrator, the uncertainty range, and cost and recall models fitted to PostgreSQL's own running times and recall on the training queries. The hook is about 660 lines of C plus the shared planner code.

Example. Query B reaches PostgreSQL as

```
SELECT id FROM items WHERE published IN (1)
  AND archive IN (4,6,9,14,21,22,27,28,30)
  ORDER BY embedding <#> '[...]' LIMIT 10
```

where the literal holds the 384 numbers of the query paper's embedding. The hook reads the vector, the two conditions and the limit, receives "search first" from the planner, and lets PostgreSQL plan with sequential and bitmap scans turned off, so `pgvector`'s iterative HNSW scan is used. For Query A it receives "filter first" and turns index scans off, so PostgreSQL filters the table and sorts the matches by distance. Queries the hook does not recognize are planned normally.

5 EVALUATION

We ask four questions. How accurate and fast is HPBS compared with histograms, exact counts, sampling and index probing (Section 5.2)? Does better estimation lead to faster queries (Section 5.3)? How much do the compare loop and each part of HVO matter (Section 5.6)? And how does the cost of wrong choices compare with the proven bounds (Section 5.6)? Every plan runs for every test query on real hardware, so the best plan is measured, not modeled. Section 6 discusses limitations.

5.1 Evaluation Settings

Hardware and software. Apple M4 (16 GB), macOS 26.3, Python 3.14 with NumPy 2.4, scikit-learn 1.8 and faiss-cpu 1.13.2, C code compiled with clang 17 (`-O3`), PostgreSQL 16.14 with `pgvector` 0.8.7.

Every latency is measured on one thread; each plan runs once as a warm-up and twice timed, and we report the slower run. PostgreSQL uses `shared_buffers`=4 GB, no parallel query or JIT, and `pgvector`'s default HNSW parameters; its latencies are client wall-clock times, including planning.

Datasets. **ARXIV-384**: one million papers from the CC0 arXiv-abstracts-2021 dump [2], embedded with all-MiniLM-L6-v2 [24] (384 dimensions, title and abstract), with real attributes (archive, published, year, numbers of categories, authors and abstract words) and a foreign key to its 172 primary categories. **AMAZON-384**: 990,000 products from Amazon Reviews 2023 [11] in four departments, embedded the same way, with real attributes (department, categories, price, rating, number of ratings, features, description length) and a foreign key to about 144,000 stores. Its filters are built from the query product, as a shopper would filter (same or another department, a price band, a minimum rating, a seller tier). **SYN-128** and **SYN-768**: one million synthetic vectors from a topic mixture, with attributes generated from the topics at graded strength. Query vectors are held-out items (real data) or noisy data vectors (synthetic). Both real benchmarks are public with exact answers [20].

Workloads. Filters combine 1–3 conditions: ranges, `IN` lists and conditions on the linked table. On SYN-128, SYN-768 and ARXIV-384 they are drawn independently of the query, so filter and query are related only through the data itself. Each benchmark has 1000 training and 1500 test queries (queries with no match are dropped); the test splits contain 630/427/413 (ARXIV-384) and 258/329/868 (AMAZON-384) anti-correlated/independent/positively correlated queries. We use $k = 10$ unless stated. Everything fitted uses only the training split.

Baselines. Estimators: PostgreSQL-style 1-D histograms with independence; the exact global count (Milvus-style); a 1% sample; and an Exqutor-style HNSW probe of the 256 nearest neighbors. Each feeds the same cost model. Routers also include the three single plans and the best global-selectivity rules (two- and three-way thresholds tuned on the training split). Inside PostgreSQL we compare `pgvector` with default settings, `pgvector`'s iterative scan (`strict_order`), Exqutor [14] built from its released code, both forced plans, and the HVO hook with and without the recall constraint. Exqutor's estimator only activates for vector *range* predicates over more than two tables, so on these queries it plans like `pgvector` 0.7.1.

Metrics and statistics. The oracle runs every plan and takes the fastest with Recall@$k \geq 0.9$; other routers are charged their plan's latency plus their planning time. Regret is a router's latency divided by the oracle's. For differences between routers we report paired 95% percentile-bootstrap intervals over test queries (2000 resamples); they capture variation across queries in one run per setting, not repeated-run timing noise.

5.2 Estimation Accuracy

Table 3 and Figure 4 report how far each estimate is from the truth, measured as the Q-error (the larger of estimate/truth and truth/estimate; 1 is perfect).

Estimators that ignore the vectors fail when filters and similarity are related, worst on real data. Histograms and exact counts both assume that the share of matches near the query equals the share in the whole table. Their 99th-percentile errors reach 765.57 and 514.26 on ARXIV-384 and 256.94 and 184.27 on AMAZON-384, against 25.96 and 26.21 on SYN-128. The errors concentrate where matches lie far from the query and where they are rare. Knowing the global share exactly does not help, because the mistake is in assuming that the local share equals it; the exact count also costs 1496 μs of planning time.

Sampling sees too little. A 1% sample (99th-percentile error 18.41 and 21.83 on the real data) contains very few matches near the query.

Probing the index works where matches are dense and fails where they are rare. The index probe is the strongest baseline. Its median error is the lowest of all estimators on every dataset (for example 1.21 on ARXIV-384), because it looks at the query's actual neighbors. When none of the first 256 neighbors match, though, it can only say "fewer than this", so its 99th-percentile error is 39.23 on ARXIV-384, and 134.39 on queries with rare local matches. It also spends 221 μs accessing the index during planning.

HPBS. The calibrated sketch has the lowest 99th-percentile error on ARXIV-384 (6.17), SYN-128 (3.08) and SYN-768 (4.31), and a version of HPBS has the lowest error on rare matches on all three (15.59 against 134.39 for the probe on ARXIV-384). AMAZON-384 is the exception: there the probe is slightly more accurate (14.86 against 16.31). Its filters are built from the query product, so matches crowd right around the query; this happens *inside* a cell, where HPBS averages it away and the probe sees it (the β term of Proposition 4.1). The calibrator mainly removes large errors: the 99th percentile falls from 9.95 to 6.17 on ARXIV-384 and from 29.86 to 16.31 on AMAZON-384. On the real data, a single map ($L = 1$) is about as accurate after calibration (6.19 and 16.83) at less than half the planning time; on synthetic data it is less accurate.

Values shared by many rows. 57% of Amazon products have price 0; giving such values their own histogram bin (in the sketch and, as a most-common-value list, in the baseline) was necessary for accurate estimates.

5.3 End-to-End Latency

Table 4 reports latency, recall and regret for every router on the real data at $k = 10$ (the synthetic-data table is in the extended version [21]). Figure 5 shows the full distribution of query times, and Figure 4(c) where the plans cross on SYN-128.

No single plan works for every query. Filtering first always pays for checking the whole table (mean 14.94 ms on ARXIV-384). Searching first is the best plan for most queries, but queries with rare local matches push its 99th percentile to 110.80 ms. ACORN-1 is rarely fastest and misses answers when matches are rare near the query (mean recall 0.619 when used for every SYN-128 query).

On real data, HVO cuts the slowest queries, and on ARXIV-384 also the average. On ARXIV-384, HVO averages 1.41 ms against 2.49 ms for the best tuned global rule (43% lower; 95% interval 34–51%), and its 99th-percentile latency is 9.19 ms against 37.08 ms. On AMAZON-384 the average is 13% lower (0.96 vs. 1.11 ms), but the interval (−4 to 28%) includes zero, so this gain is not established with our number of queries; the gain on the slowest queries is clear (7.00 vs. 18.65 ms, Figure 5b). HVO's recall equals the global rule's (0.988 vs. 0.989 on ARXIV-384), and its latency is close to that of

Table 3: Local-selectivity Q-error on the test split, by correlation regime and for rare local matches ($s_{\text{loc}} < 10^{-3}$), and median planning-time cost of the estimator. Lower is better.

	Anti-correlated			Independent			Positively correlated			Rare	Time
Estimator	P50	P95	P99	P50	P95	P99	P50	P95	P99	P99	(μs)
ARXIV-384 (real)											
1-D hist. + AVI (PG-style)	7.49	459.83	1265.85	1.23	3.42	17.65	1.67	7.31	24.04	2002.20	3
Exact s_g, s_{loc}=s_g (Milvus-style)	6.23	238.56	1133.30	1.21	1.63	1.76	1.71	5.43	18.67	1312.87	1496
Uniform sample 1%	2.80	10.32	30.67	1.80	6.27	11.58	1.30	3.82	7.97	53.68	255
HNSW probe, P=256 (Exqutor-style)	1.46	17.95	45.71	1.16	1.89	18.39	1.12	1.75	2.51	134.39	221
HPBS, cross-polytope cells	5.26	55.64	151.79	6.77	21.34	26.40	21.23	31.46	36.74	287.10	135
HPBS raw (L=4, m=1024)	1.44	4.63	8.07	1.97	5.23	6.41	4.67	8.76	11.02	23.29	134
HPBS, L=1 + calibrator	1.54	4.02	7.80	1.24	2.16	2.78	1.51	3.42	5.36	13.87	57
HPBS + calibrator	1.52	4.02	7.03	1.28	2.23	3.39	1.53	2.95	6.19	15.59	138
AMAZON-384 (real)											
1-D hist. + AVI (PG-style)	3.56	120.43	418.10	1.30	3.60	223.54	3.91	37.19	160.15	678.47	9
Exact s_g, s_{loc}=s_g (Milvus-style)	3.18	146.50	348.08	1.18	1.65	1.76	3.58	25.57	75.94	504.03	1462
Uniform sample 1%	2.62	10.30	29.04	1.64	8.31	18.49	1.41	7.40	20.88	27.81	243
HNSW probe, P=256 (Exqutor-style)	1.43	8.82	39.38	1.17	3.58	20.96	1.19	2.39	5.32	142.99	171
HPBS, cross-polytope cells	3.69	36.79	71.00	8.03	36.49	151.81	21.75	42.86	227.74	904.05	135
HPBS raw (L=4, m=1024)	1.54	6.28	10.52	2.38	9.32	38.81	4.53	11.88	45.50	54.24	128
HPBS, L=1 + calibrator	1.85	9.46	17.20	1.40	5.78	22.71	1.49	5.36	9.70	30.76	55
HPBS + calibrator	1.80	7.17	20.98	1.49	6.05	25.44	1.52	4.48	10.65	36.24	132
SYN-128											
1-D hist. + AVI (PG-style)	3.01	12.16	73.43	1.24	1.74	1.89	2.19	8.15	13.46	169.62	11
Exact s_g, s_{loc}=s_g (Milvus-style)	2.97	14.02	73.38	1.23	1.68	1.78	2.20	8.19	11.13	169.67	1517
Uniform sample 1%	2.22	11.20	28.69	1.90	9.10	18.98	1.59	6.40	14.14	133.81	80
HNSW probe, P=256 (Exqutor-style)	1.27	3.69	6.73	1.16	2.17	3.24	1.16	1.71	2.56	25.53	113
HPBS, cross-polytope cells	1.79	10.96	27.67	3.37	18.81	23.98	14.31	30.87	36.26	59.82	124
HPBS raw (L=4, m=1024)	1.24	2.33	4.89	1.68	5.70	8.59	3.35	9.75	13.54	1.76	82
HPBS, L=1 + calibrator	1.32	2.44	3.72	1.25	2.09	3.20	1.53	3.95	7.20	2.38	43
HPBS + calibrator	1.29	2.13	2.85	1.26	1.99	2.77	1.41	2.65	3.96	2.30	86
SYN-768											
1-D hist. + AVI (PG-style)	2.98	14.77	58.43	1.24	1.76	1.93	2.28	7.20	18.72	136.61	11
Exact s_g, s_{loc}=s_g (Milvus-style)	2.96	16.38	58.94	1.23	1.72	1.80	2.30	6.83	19.01	139.95	1520
Uniform sample 1%	2.22	11.33	32.87	1.82	9.34	20.42	1.60	6.55	15.31	10.88	529
HNSW probe, P=256 (Exqutor-style)	1.27	4.67	7.71	1.16	1.92	3.76	1.17	1.88	2.27	12.97	169
HPBS, cross-polytope cells	1.92	10.79	22.05	3.41	18.90	27.48	11.76	29.96	35.77	58.01	134
HPBS raw (L=4, m=1024)	1.25	2.78	6.90	1.81	8.39	13.51	3.66	13.76	18.60	1.86	136
HPBS, L=1 + calibrator	1.35	2.60	4.75	1.25	2.06	2.45	1.60	5.17	8.49	2.20	51
HPBS + calibrator	1.33	2.34	3.55	1.26	1.96	2.75	1.60	4.20	5.99	2.08	140

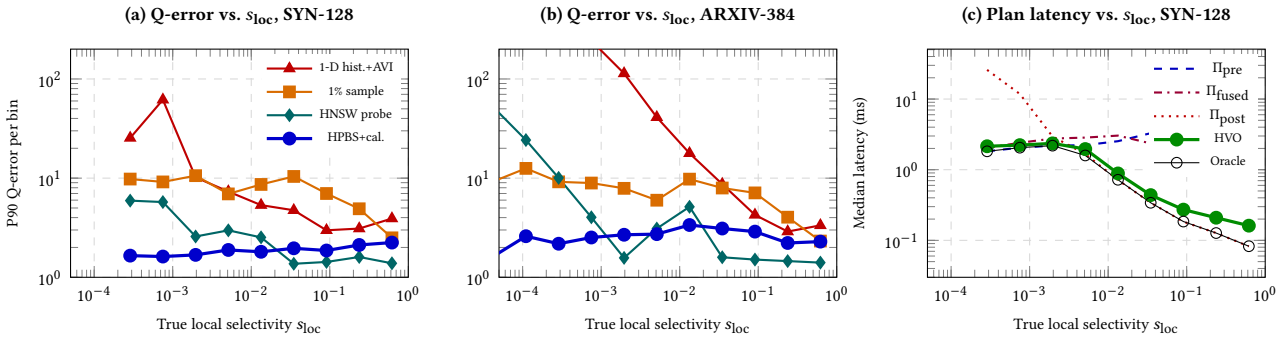


Figure 4: (a, b) P90 Q-error within bins of true local selectivity: the baselines degrade as matches become rare near the query, and HPBS does not. (c) Median latency of each plan by true local selectivity on SYN-128, with HVO (including planning time) and the oracle. Pre-filtering is flat in s_{loc}; the graph plans' cost explodes as s_{loc} falls.

the same cost model given the true selectivities (1.14 and 0.62 ms). The oracle's average (1.28 and 0.86 ms) is sometimes *higher* than HVO's, because the oracle refuses plans that miss answers.

On synthetic data, HVO ties the global rule on average (both intervals include zero, Figure 6) and wins on the slowest queries. HVO averages 0.73 and 1.19 ms against 0.72 and 1.20 ms, with 99th percentiles of 3.21 vs. 5.69 ms and 5.57 vs. 7.40 ms. Searching first works well for most of these queries, so a single threshold already gets most of them right; the remaining gain comes from the few queries it gets wrong, and HVO's planning time offsets it. With a single map ($L = 1$, Table 6), HVO is faster than the global rule on

Table 4: End-to-end routing on the real datasets ($k = 10$): latency (ms, including planning time), mean Recall@10, share of queries with Recall@10 < 0.9, plan accuracy against the oracle, measured regret, and median planning time (μs). CM: the calibrated cost model of Section 4.4; "robust": minimax routing with the interval width fitted on the training split.

Router	Mean	P50	P95	P99	Recall	Low (%)	Acc. (%)	Mean reg.	P99 reg.	Plan (μs)
ARXIV-384 (real)										
Always pre-filter	14.94	11.20	36.57	41.30	1.000	0.0	18.0	82.59	395.69	0
Always post-filter (resumable)	7.83	0.26	65.50	110.80	0.966	6.2	76.7	4.05	62.10	0
Always fused (ACORN-1)	5.40	2.54	16.13	42.59	0.930	14.5	5.2	15.64	91.12	0
Global s_g threshold	2.49	0.28	5.98	37.08	0.989	2.3	79.9	2.12	22.82	0
CM + 1-D histograms	4.78	0.27	25.11	97.70	0.978	4.7	81.6	2.26	30.80	3
CM + HNSW probe	1.55	0.56	5.89	9.83	0.984	3.1	89.4	1.98	4.28	221
CM + HNSW probe, robust	1.62	0.56	6.83	10.43	0.987	2.4	86.9	2.06	4.61	221
HVO, point routing	1.34	0.40	5.57	9.32	0.985	2.9	91.3	1.54	2.79	138
HVO (robust)	1.41	0.40	6.07	9.19	0.988	2.4	89.9	1.58	3.21	138
HVO, recall-aware	1.47	0.40	6.40	9.91	0.991	1.7	90.1	1.60	3.38	138
CM + true selectivities	1.14	0.26	5.10	8.10	0.987	2.5	92.7	1.01	2.01	0
Oracle	1.28	0.27	5.31	10.36	0.992	0.0	100.0	1.00	1.00	0
AMAZON-384 (real)										
Always pre-filter	12.63	8.15	38.37	42.79	0.999	0.0	10.8	89.34	459.48	0
Always post-filter (resumable)	2.69	0.19	9.45	74.40	0.977	4.3	87.5	1.99	29.75	0
Always fused (ACORN-1)	7.55	2.16	7.95	38.27	0.903	16.2	1.7	14.74	70.55	0
Global s_g threshold	1.11	0.20	2.82	18.65	0.983	3.5	87.6	1.47	13.71	0
CM + 1-D histograms	1.92	0.21	3.95	38.32	0.981	3.9	86.6	1.98	28.36	9
CM + HNSW probe	0.90	0.39	3.06	6.97	0.987	2.5	93.9	1.97	3.47	171
CM + HNSW probe, robust	0.93	0.39	3.24	7.40	0.989	2.3	93.0	2.00	3.63	171
HVO, point routing	0.97	0.35	3.24	7.16	0.985	3.0	93.3	1.92	3.38	132
HVO (robust)	0.96	0.36	3.30	7.00	0.988	2.5	91.1	2.00	6.54	132
HVO, recall-aware	1.01	0.36	3.62	7.92	0.989	2.2	90.7	2.05	6.85	132
CM + true selectivities	0.62	0.19	2.57	6.09	0.986	3.0	95.1	0.99	1.21	0
Oracle	0.86	0.20	3.08	10.16	0.994	0.0	100.0	1.00	1.00	0

Table 5: Inside PostgreSQL 16 + `pgvector` on the real datasets ($k = 10$, test split): client latency in ms (including planning), mean Recall@10, and share of queries with Recall@10 < 0.9. The oracle takes the faster forced plan that meets Recall@10 ≥ 0.9 (pre-filtering if neither does). Exqutor was run on ARXIV-384 only; it needs its own patched PostgreSQL build and a separate copy of the data.

	ARXIV-384					AMAZON-384				
Configuration	Mean	P50	P99	Recall	Low (%)	Mean	P50	P99	Recall	Low (%)
pgvector 0.8.7, defaults	0.60	0.51	1.47	0.540	56.8	0.46	0.27	8.03	0.686	40.5
pgvector 0.8.7, iterative scan	5.09	0.60	27.21	0.846	28.1	2.20	0.35	26.24	0.904	16.8
Exqutor (pgvector 0.7.1 + patches)	0.30	0.28	0.49	0.538	56.1	–	–	–	–	–
Forced pre-filter	115.86	139.62	189.00	1.000	0.0	90.29	79.49	213.70	0.999	0.0
Forced iterative post-filter	18.04	0.67	214.62	0.907	20.7	5.69	0.41	153.83	0.931	14.1
HVO hook	16.10	0.80	172.03	0.953	10.5	4.45	0.64	51.04	0.953	9.4
HVO hook, recall-aware	50.00	1.21	184.58	0.984	3.5	43.64	4.97	229.54	0.988	2.2
Oracle	24.17	0.71	181.95	0.986	0.0	10.16	0.41	164.87	0.991	0.0

all four datasets, and all four 95% intervals exclude zero (gains 48, 25, 10 and 11%; lower ends 39, 10, 2 and 4%).

Which queries are routed differently. HVO and the global rule send similar shares of queries to each plan; the difference is *which* queries go where. Both match the fast search-first plan on most queries, but HVO avoids the wrongly routed rare-match queries that make up the global rule's slowest cases (Figure 5).

Back to the running example. The pair in Figure 1 was chosen to show the problem, so it is fair to ask how often it happens. On the ARXIV-384 test queries, HVO and the best global rule choose different plans for 15% of queries, and on those queries HVO's choice (including its planning time) is faster 89% of the time. The global rule's choice differs from the oracle's on 20% of queries, and HVO's on 10%. On AMAZON-384 they disagree less often (9%), HVO is faster on 70% of those queries, and the two rules differ from the oracle on 12% and 9% of queries, which fits the smaller and less certain gain there.

Good costs need good estimates, and index probing is the closest competitor. Given histogram estimates, the same cost model averages 4.78 ms on ARXIV-384. Given the index probe, it averages 1.55 and 0.90 ms (1.62 and 0.93 ms with minimax routing). That is behind HVO on ARXIV-384, where rare matches dominate the slowest queries, and slightly *ahead* of HVO on AMAZON-384 (0.96 ms), where the probe estimates better; HVO with a single map (0.83 ms) is ahead of both. HVO's advantages over probing are that it never touches the index during planning and that it stays accurate when matches are rare.

Recall-aware routing. Allowing graph plans only when the recall model predicts at least 9 of the top 10 raises recall to 0.991 and 0.989 on the real data, at 1.47 and 1.01 ms. It mostly avoids ACORN, which misses many answers when matches are rare near the query.

What still limits HVO. Planning time (the native planner takes 90–89 μs, which matters when the best plan takes a few hundred microseconds) and the cost model's own error (a median factor of

1.19 for the best plan on ARXIV-384) explain most of the remaining gap to the cost model with true selectivities.

5.4 Inside PostgreSQL

Table 5 runs the same test queries as SQL in PostgreSQL with `pgvector`.

Default `pgvector` *and Exqutor are fast because they return incomplete answers.* With default settings, `pgvector`'s HNSW scan stops after $ef_{\text{search}} = 40$ candidates. For filtered queries it then returns fewer than k rows, or rows outside the true top k: mean recall 0.540 on ARXIV-384, with 56.8% of queries below 0.9. Exqutor behaves the same way (recall 0.538): as its design intends, its estimator does not activate on single-table top-k queries.

`pgvector`*'s iterative scan improves recall, but not enough.* With `strict_order`, recall rises to 0.846 and 0.904, but 28.1% and 16.8% of queries still fall below 0.9. The scan gives up after `max_scan_tuples`=20,000 rows, which queries with far-away matches use up.

HVO in PostgreSQL. The hook sends each query either to filter first (filter the table, then sort by distance) or to the iterative scan with a larger row budget. Compared with forcing that iterative scan for every query, it is both faster and more accurate on both datasets: 16.10 vs. 18.04 ms with recall 0.953 vs. 0.907 on ARXIV-384, and 4.45 vs. 5.69 ms with recall 0.953 vs. 0.931 on AMAZON-384. Its average is below the oracle's (24.17 and 10.16 ms), whose recall target forces many slow filter-first plans. Planning in the hook takes 126 and 119 μs (median). With the recall constraint, HVO reaches recall 0.984 and 0.988, close to the oracle's (0.986 and 0.991). In PostgreSQL that is expensive (mean 50.00 and 43.64 ms), because filtering first there reads the table and computes distances row by row: a median of 139.62 ms, against 11.20 ms for our column-wise loop.

In PostgreSQL the choice is therefore a trade-off between speed and completeness, not a single speedup. The setups most users would run are either fast and incomplete (defaults) or still incomplete on 17–28% of queries (iterative scan). HVO offers two settings that both beat forcing the iterative scan.

5.5 Effect of k

Figure 6 varies k. At $k = 100$, HVO's average is 30% and 27% below the global rule's on the real data, and its 99th-percentile regret is 2–10× lower on every dataset. At $k = 1$ the picture reverses on three of four datasets: single-result queries are so cheap that planning time dominates, and both HPBS and the probe estimate poorly, because the answer depends on a single match. HVO is then slower than the threshold rule except on ARXIV-384 (0.83 vs. 1.38 ms). Routing pays off once queries cost more than planning. The 95% intervals in Figure 6 show the gain is clearly positive on ARXIV-384 at every k and on AMAZON-384 at $k = 100$, while the intervals for SYN-128 and SYN-768 at $k = 10$ and $k = 100$ and for AMAZON-384 at $k \leq 10$ include zero. At $k = 1$, HVO is clearly slower than the global rule on SYN-768.

5.6 Ablations, Transfer and Regret

Removing parts of HVO. Table 6 removes one part at a time:

- *Counted instead of measured costs.* Counting distance computations treats a sequential and a random computation as equally expensive. HVO's average rises from 1.41 to 2.25 ms on ARXIV-384, from 0.96 to 5.19 ms on AMAZON-384 and from 0.73 to 4.60 ms on SYN-128. Counting ignores that random accesses cost far more than sequential ones and that ACORN must first evaluate the filter, so it sends many queries to ACORN.
- *No calibrator.* HVO is slower on every dataset (1.55, 1.17, 0.83 and 1.50 ms), because the uncorrected sketch makes more large errors and therefore needs a wider safety range.
- *Point vs. minimax routing.* With HPBS's actual errors, the two differ by a few percent in mean and 99th-percentile latency (Table 4). Minimax routing's value here is its guarantee (Theorem 4.4), not a measured speedup.
- *Width of the safety range.* No width is best on every dataset. Widths covering up to 95% of training errors stay within about 15% of one another in average latency, and point routing has the lowest average on ARXIV-384. The widest range (99%) is the slowest on all four datasets, because it sends too many queries to filter first. We keep the 90% width, fixed before looking at test data, for its guarantee rather than for a measured speedup.

Calibrator transfer. Trained only on SYN-128, the calibrator brings the 99th-percentile error on ARXIV-384 to 5.82, against 6.17 when trained on ARXIV-384 itself and 9.95 without one: it learns the sketch's systematic biases, which are largely the same across datasets.

Regret under injected error. Replacing the estimate on SYN-128 by the truth multiplied or divided by η (20 repetitions per η), the model's regret never exceeded the bounds of Lemma 4.2 and Theorems 4.3–4.4 across 300000 routed queries (0 violations), and minimax routing had zero regret on every query whose best plan was filtering first. The bounds are loose: at $\eta = 1.1$ the bound is 5.30, against an observed 99th-percentile regret of 1.00.

6 DISCUSSION

6.1 Limitations

Data and workloads. Two of the four datasets are real, but none of the filter workloads comes from real query logs, because no public log of filtered vector queries exists. The arXiv filters are random, and the Amazon filters are derived from the query product. HVO's advantage was larger on real data than on synthetic data, which suggests that real data relates filters and similarity more strongly than our generator does. On the Amazon workload, though, index probing estimated better than HPBS. Relationships inside a cell are the sketch's blind spot (Proposition 4.1), and filters built from the query itself create exactly those.

One machine, one graph library. All results come from one Apple M4 with ARM NEON code; absolute times and fitted constants must be re-measured on each machine, while the form of the cost model carries over. ACORN, used as the reference implementation, missed many answers when matches were rare near the query; a better-tuned ACORN would strengthen three-way routing. In PostgreSQL the hook steers the planner with standard settings, `pgvector` cannot filter while searching, and its filter-first plan is

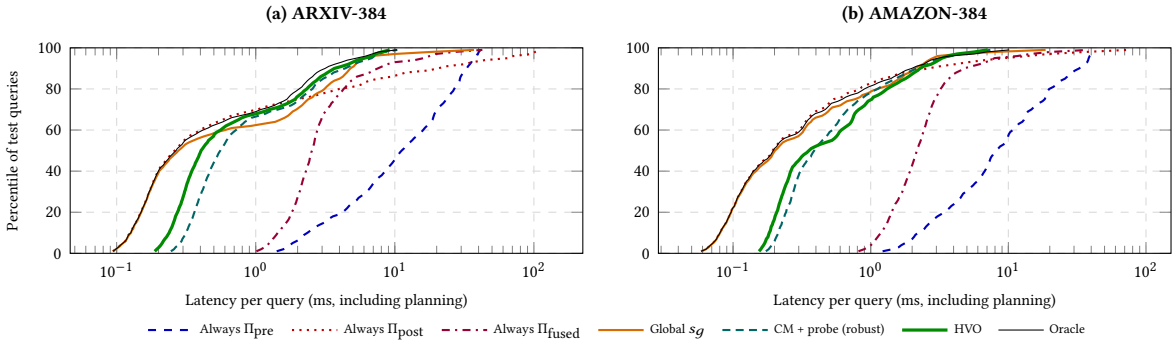


Figure 5: Distribution of per-query latency on the real test splits ($k = 10$; percentiles 1–99, logarithmic latency axis in ms, including planning time). Curves further left are faster. "Global s_g": best tuned global-selectivity router; "CM + probe": the calibrated cost model fed the HNSW probe, with minimax routing. The always-post curve has the fastest body and the slowest tail; HVO follows the faster plans in the body and cuts the tail.

Table 6: Ablations of HVO ($k = 10$): mean latency (ms), mean Recall@10, and P99 measured regret.

	ARXIV-384			AMAZON-384			SYN-128			SYN-768		
Variant	Mean	Rec.	P99 reg.	Mean	Rec.	P99 reg.	Mean	Rec.	P99 reg.	Mean	Rec.	P99 reg.
HVO (robust, L=4, calibrated)	1.41	0.988	3.21	0.96	0.988	6.54	0.73	0.981	3.27	1.19	0.967	2.79
point routing (no interval)	1.34	0.985	2.79	0.97	0.985	3.38	0.73	0.980	3.22	1.18	0.966	2.79
+ recall constraint	1.47	0.991	3.38	1.01	0.989	6.85	0.73	0.981	3.27	1.29	0.967	3.07
without calibrator	1.55	0.989	5.06	1.17	0.990	20.23	0.83	0.983	5.02	1.50	0.969	7.09
L=1 partition	1.31	0.988	3.03	0.83	0.989	6.05	0.65	0.981	2.04	1.07	0.967	1.77
counted cost (distance evaluations)	2.25	0.972	11.78	5.19	0.981	17.71	4.60	0.709	181.02	6.40	0.750	92.04
+ runtime early-exit guard	1.41	0.988	3.21	0.96	0.988	6.54	0.73	0.981	3.27	1.19	0.967	2.79
interval q=0.75	1.37	0.987	2.84	0.89	0.987	3.59	0.72	0.981	3.26	1.17	0.967	2.77
interval q=0.95	1.44	0.988	3.31	1.02	0.988	12.00	0.74	0.981	3.29	1.20	0.967	2.79
planning time excluded	1.24	0.988	3.06	0.74	0.988	5.93	0.59	0.981	1.69	1.00	0.967	1.52

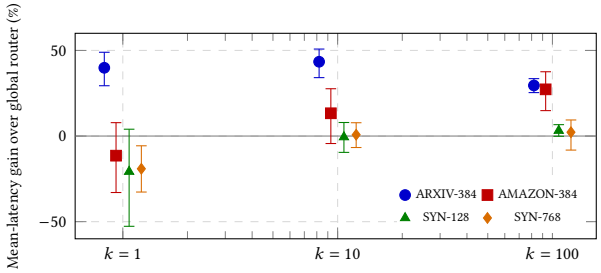


Figure 6: Relative mean-latency gain of HVO over the best tuned global-selectivity router, $1 - \overline{\text{lat}}_{\text{HVO}}/\overline{\text{lat}}_{\text{global}}$, with paired 95% bootstrap intervals over test queries (2000 resamples). Points are offset horizontally per dataset for legibility. Intervals that cross zero indicate no detectable difference.

slow, which makes the recall-aware setting expensive. The 95% intervals cover variation across queries in one run per setting.

6.2 When to Use HVO

The results suggest a few practical rules. (1) Route only when it can pay off: if the cheapest plan is predicted to cost less than planning itself (small k, very selective or very broad filters), skip HVO and use the default plan. (2) Measure costs on the machine that will run the queries: the cost constants come from forced runs of the training queries (1000 per dataset here), while the calibrator transfers reasonably across datasets (Section 5.6). (3) Inside PostgreSQL, first decide whether the application needs complete answers: the recall-aware setting returns almost the oracle's recall but pays for PostgreSQL's slow filter-first plan, while the default HVO setting is faster and still more accurate than forcing the iterative scan.

7 CONCLUSION

Choosing a plan for a filtered vector query requires knowing how far the search must travel before it finds k matches. We defined that distance as the k-th-match local selectivity, estimated it with a compact summary that never touches the index, and turned the estimate into a plan choice with measured costs and proven guarantees. HVO cuts average latency on arXiv by 43% (95% interval 34–51%) and the slowest queries several-fold on both real datasets, though its average gain on Amazon and synthetic data is not statistically clear, and inside PostgreSQL it returns more correct results than forcing `pgvector`'s iterative scan while being faster. Testing other machines, graph libraries and real query logs is future work.

REFERENCES

[1] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu. 2026. Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis. arXiv:2602.11443.

[2] arxiv-abstracts-2021 2021. arxiv-abstracts-2021: Metadata and Abstracts of arXiv Papers. Hugging Face dataset `gfissore/arxiv-abstracts-2021`, CC0 license.

https://huggingface.co/datasets/gfissore/arxiv-abstracts-2021.

[3] K. Beyer, P. J. Haas, B. Reinwald, Y. Sismanis, and R. Gemulla. 2007. On Synopses for Distinct-Value Estimation under Multiset Operations. In *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data*. 199–210.

[4] M. S. Charikar. 2002. Similarity Estimation Techniques from Rounding Algorithms. In *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*. 380–388.

[5] M. Douze et al. 2024. The Faiss library. arXiv:2401.08281.

[6] DuckDB. 2024. Vector Similarity Search Extension (vss). https://duckdb.org/docs/extensions/vss.

[7] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier. 2007. HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm. In *Proceedings of the 2007 Conference on Analysis of Algorithms (AofA)*. 127–146.

[8] S. Gollapudi et al. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.

[9] D. Gunopulos, G. Kollios, V. J. Tsotras, and C. Domeniconi. 2000. Approximating Multi-Dimensional Aggregate Range Queries over Real Attributes. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. 463–474.

[10] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, not from Queries! *Proc. VLDB Endow.* 13, 7 (2020), 992–1005.

[11] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. arXiv:2403.03952.

[12] P. Iff et al. 2025. Benchmarking Filtered Approximate Nearest Neighbor Search Algorithms on Transformer-based Embedding Vectors. arXiv:2507.21989.

[13] A. Kane. 2024. pgvector: Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector.

[14] H. Kim, C. Lim, H. An, R. Sen, and K. Park. 2026. Exqutor: Extended Query Optimizer for Vector-Augmented Analytical Queries. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. arXiv:2512.09695; code: https://github.com/BDAI-Research/Exqutor.

[15] A. Kipf et al. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *9th Biennial Conference on Innovative Data Systems Research (CIDR)*.

[16] H. Lan, S. Huang, Z. Bao, and R. Borovica-Gajic. 2024. Cardinality Estimation for Similarity Search on High-Dimensional Data Objects: The Impact of Reference Objects. *Proc. VLDB Endow.* 18, 3 (2024), 544–556.

[17] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang. 2025. Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data. arXiv:2505.06501.

[18] Y. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[19] Y. Matias, J. S. Vitter, and M. Wang. 1998. Wavelet-Based Histograms for Selectivity Estimation. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*. 448–459.

[20] Sanjay Mishra. 2026. Correlated Filtered Vector Search Benchmark (ARXIV-384, AMAZON-384). Hugging Face dataset `sanmish4/correlated-filtered-vector-search`. https://huggingface.co/datasets/sanmish4/correlated-filtered-vector-search.

[21] Sanjay Mishra. 2026. It's Not How Many, It's Where: Optimizing Routing Selection for Filtered Vector Search (Extended Version). https://github.com/sanmish4ds/HyperVectorOptimizer/blob/main/paper/hvo.pdf.

[22] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3 (2024).

[23] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 10 (2025), 3256–3268.

[24] N. Reimers and I. Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of EMNLP-IJCNLP 2019, 3982–3992.

[25] J. Shi, Y. Cai, and W. Zheng. 2025. Filtered Approximate Nearest Neighbor Search: A Unified Benchmark and Systematic Experimental Study. arXiv:2509.07789.

[26] R. Spring and A. Shrivastava. 2017. A New Unbiased and Efficient Class of LSH-Based Samplers and Estimators for Partition Function Computation in Log-Linear Models. arXiv:1703.05160.

[27] J. Wang et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627.

[28] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni. 2022. Navigable Proximity Graph-Driven Native Hybrid Queries with Structured and Unstructured Constraints. arXiv:2203.13601.

[29] C. Wei et al. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proc. VLDB Endow.* 13, 12 (2020), 3152–3165.

[30] W. Wu et al. 2022. HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*. 4580–4584.

[31] Z. Yang et al. 2020. NeuroCard: One Cardinality Estimator for All Tables. *Proc. VLDB Endow.* 14, 1 (2020), 61–73.

[32] Q. Zhang et al. 2023. VBase: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 377–395.

[33] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024).